Autonomous Robotic Systems

Florida Polytechnic University

Final Report – Autonomous Car Robot

Submitted by:
Lyam Paulette Barreto
Michael Beleut
Felipe Gonzalez

Submitted to:
Dr. Huan Ngo

Date:
May 5th 2026

Table of Contents

Abstract

This report details the development and optimization of the Goosebot, an autonomous ground vehicle engineered for reliable lane-following within a constrained track environment. The system incorporates a 3D-printed chassis, a dual-motor differential drive system, L298N motor drivers, and pulse-width modulation (PWM)-based control to achieve stable and responsive navigation. A regulated power architecture utilizing a 12V supply with DC-DC conversion supports safe and consistent operation of onboard electronics.

The perception pipeline utilizes a customized YOLO11 model. Although the initial design incorporated detection of obstacles such as rubber duckies and red stop lines, the final competition strategy prioritized optimized lane-following. Consequently, the model was simplified to focus exclusively on white and yellow lane markings, thereby improving efficiency and reducing computational overhead.

The software architecture was further modified to incorporate a multithreading approach for task allocation, which enables concurrent execution of perception, control, and decision-making processes. This modification enhanced real-time responsiveness and improved overall system stability. Data collection and augmentation techniques were implemented to enhance model robustness under varying environmental conditions.

Introduction

Autonomous ground vehicles must integrate mechanical design, embedded control systems, and machine learning-based perception to navigate reliably in structured environments. In track-based applications like the Goosebot, consistent lane-following is essential for stability and course completion. The main challenge is to develop a system that accurately interprets visual lane information and delivers responsive, stable motor control in real time.

The Goosebot was initially developed to perform both lane navigation and obstacle-based decision making. Early perception models detected dynamic obstacles, such as rubber duckies and red stop lines, prompting the vehicle to halt or react as needed. During competition development, other teams adopted alternative strategies for obstacle handling, which shifted the final implementation focus to optimizing lane-following performance.

The final system uses a vision-based lane detection pipeline with a dual-motor differential drive. The mechanical platform features a 3D-printed chassis and four DC motors controlled by L298N H-bridge drivers. PWM signals provide precise speed and direction control for steering and trajectory correction. A regulated power system uses a 12V supply and a DC-DC converter to step down to 5V for sensitive electronics, ensuring stability and protecting components.

The perception system uses a customized YOLO11 deep learning model trained for lane detection. To align with competition requirements, the model was simplified, and the drive.py code was modified to better align with the competition criteria. This streamlining improves computational efficiency and enables the model to focus on relevant navigation features. This report describes the mechanical/electrical assembly, code modification, and machine learning-based perception approach used in developing the Goosebot.

Materials and Methods

Materials and Methods: Components and Physical Assembly

The implementation of the Goosebot first required the physical assembly of the robot. The materials involved in this assembly are as listed below:

- 2 GB Radxa Rock 5C
- 2x DC motors
- 3 printed chassis
- 4x wheels
- 1080p camera
- 2x L298N motor drivers
- 5 V DC Converter
- PCA9685 PWM board
- Heat inserts and screws
- 12 V LiPo Battery (2200mAh - 50C)

Their assembly is as shown on the diagram below, with proper wiring between the single-board computer and the peripheral systems.

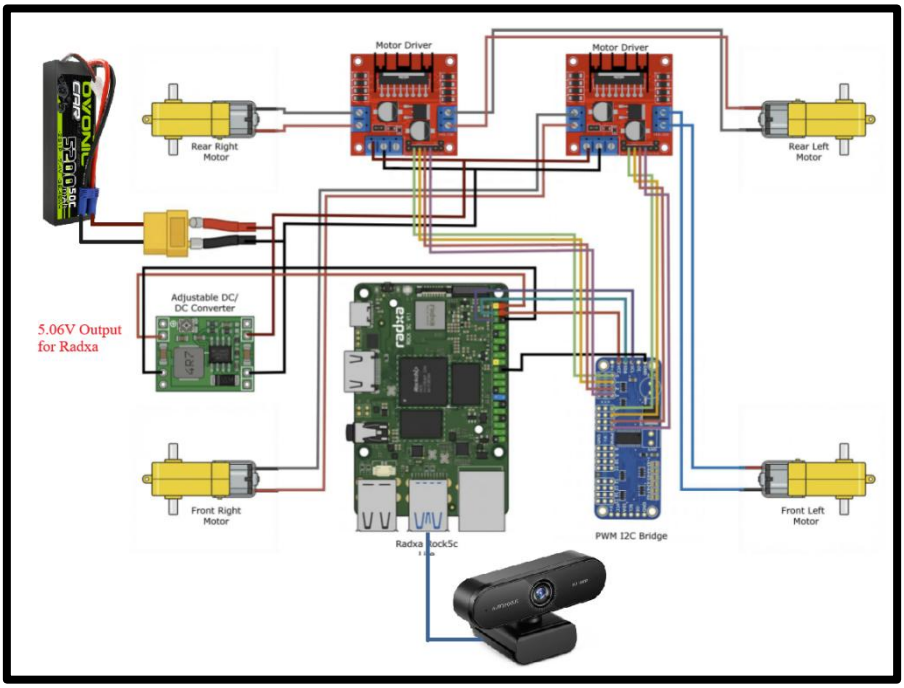

Figure 1. Wiring Diagram

Materials and Methods: Detection Model

The Goosebot's consistency and lane accuracy were enhanced by redesigning the perception pipeline with a customized YOLO11-based model. Given that the competition environment did not include dynamic obstacles such as stop lines or barriers, the model was optimized to prioritize the most critical visual features for navigation: white and yellow lane markings.

Focusing exclusively on these features led to increased detection accuracy and confidence. The dataset was updated with new images of the track under typical operating conditions and supplemented with previously collected samples to maintain variability. Annotation methods were refined to improve clarity and reduce ambiguity. Bounding boxes were minimized and restricted to essential features and overlap between labeled regions was eliminated to prevent misclassification during training.

Model efficiency was improved by removing unnecessary classes related to obstacles absent from the race environment. Only two classes were retained: white lane markings and yellow lane markings. This reduction in class complexity decreased memory usage and enabled greater allocation of computational resources to learning lane-specific features. As a result, inference speed increased and real-time performance on embedded hardware improved.

Model robustness was strengthened by applying data augmentation techniques to expand the dataset. The base dataset increased from 42 to 60 images and was further augmented through systematic variations in saturation, brightness, and other visual transformations. The final dataset consisted of 112 images, including 104 training images, 5 validation images, and 3 testing images.

Evaluation of model performance demonstrated strong classification results across both validation and test sets. On the validation set, overall accuracy reached 72%, with class-specific performance of 64% for white lane markings and 79% for yellow lane markings. On the test set, overall accuracy was 65%, with 58% accuracy for white lane markings and 72% for yellow lane markings. In general image identification scenarios, the model achieved an average detection accuracy of approximately 86%, with yellow lane marking detection reaching up to 91%. These results indicate stronger performance for yellow lane markings across varied conditions.

After the YOLO11-based lane detection model was trained, the model files had to be converted into RKNN format so they could run efficiently on the Radxa Rock 5C. This conversion was necessary because the original AI model weights, such as .pt, or ONNX model files, are not optimized to run directly on the Rock 5C's neural processing unit. Instructor Hoan Ngo's repository explains that, for the model to be executed on the Rock 5C's NPU rather than only the CPU, the ONNX model must be converted into RKNN format [1]. This is important because running the model through the NPU allows the robot to perform real-time autonomous

detection efficiently, which is important for lane following where any sort of late detections can cause unstable steering or missing turns.

The RKNN conversion instructions used for this project are in Ngo's Goose GitHub repository under the "05_npu_conversion" folder [1]. In this section, Ngo provides the setup process for converting a YOLO11 model into a Rockchip-compatible RKNN model. This process includes preparing an Ubuntu 22.04 or WSL2 Ubuntu 22.04 environment, installing Python tools, creating a virtual environment, cloning RKNN Toolkit 2, and exporting the trained YOLO model into RKNN format. The trained model is converted using the Ultralytics export command, which outputs an RKNN model folder that can be copied onto the Rock 5C and loaded by the driving.py script.

This conversion step connected the trained AI model to the final embedded system. Once converted, the RKNN model could be loaded into the Goosebot's implementation script as "best_goose_rknn_model," allowing the Goosebot to perform real-time lane detection during its autonomous driving. Without this conversion, the robot would likely depend on the CPU, which could reduce performance and make lane-detection less stable during testing.

This targeted optimization approach enhanced lane detection reliability, trajectory stability, and overall consistency in autonomous navigation within the constraints of the track.

Materials and Methods: Implementation Script

The implementation script test_drive_v3.py [2] features web stream of camera feed visualizing lane detection, estimation of lane center, steering behavior dictated through PD control, and safe stop if detection freezes or if 'Ctrl+ C' is pressed. First, the program separates vision and motor control into two individual threads labeled by the vision_stream_loop() and drive_loop() functions, respectively. The separation of these constant and memory-intensive processes supports their simultaneous execution without blocking one another. No more than two separate threads were created in order to not overload the available RAM.

The stream and detection loop initialize the Flask web server and the video capture to be integrated using OpenCV. The detection functionality is dictated by the trained YOLO model discussed in the Detection Model subsection of this Materials and Methods section of the report which, after undergoing RKNN conversion, can be loaded from the MODEL_PATH variable on the script – the model having been saved as 'best_goose_rknn_model'. The detection patterns learned by the model are stored in a DetectionState data class object, where the latest detection data – like x-coordinate of the best yellow and white line detection, where the robot wants the lane center to be (target_x), steering command, among other things – are updated for proper and current reaction of the robot. The prediction algorithm looks for two classes: 'yellow line' and 'white line'. For each detected box, it receives center x-position of the bounding box (x), center

y-position of the bounding b ox (y), box width (w), and box height (h). It then chooses the largest yellow line and largest yellow line based on area, while ignoring detections that are too high in the frame, as seen in the code section below.

```
if y < cutoff_pixel:
     continue
area = w * h
```

Therefore, the script chooses the largest detection. Target selection, then, works by leading the robot to aim for the midpoint between the yellow and white line if both are detected, assuming that the lane center is half a lane width to the left of the yellow or white line, whichever is detected on its own. If no lane is seen, however, the robot is set to assume the lane center is straight ahead. From this logic, the steering is calculated and sent over to the drive command thread loop.

The drive command loop initializes I2C communication between the PWM controlling the motors and the Radxa, maps the motors to their respective channels, and commands the motors to either charge forward, slow down, or stop. The channel configuration and speed setting is performed by creating object instances from a data class defined as Motor. While the target location and related steer values are calculated in the detection loop, the drive command loop executes the motion by sending it to the motors. The calculation implementing PD control to determine the steering value to be sent to the motors can be seen below, as also seen in the vision_stream_loop() thread.

```
# PD steering
error = target_x - CENTER_X
derivative = error - prev_error
prev_error = error


steering = (error * Kp) + (derivative * Kd)
steering = max(min(steering, MAX_STEER), -MAX_STEER)
```

The MAX_STEER variable sets a physical limit to the steer speed, set as 0.75 on the test_drive_v3.py. The proportional term of the PD control – which reacts to how far the lane center is from the camera center - was kept at 0.0007 and the derivative term – which reacts to quickly the error is changing – to 0.0009. Now, in this same thread, the motors are set to crawl at a speed of 0.1 their maximum speed when no lane is seen or the model loses detection momentarily, defined by the variable NO_LANE_SPEED, in order to allow the model to slow down before steering incorrectly. The normal speed defined in the BASE_SPEED variable, on

the other hand, was set to 0.23 of the maximum possible speed, which was the speed found to open room for stability while supporting a fast ride.

The diagram below summarizes the software structure of the Goosebot.

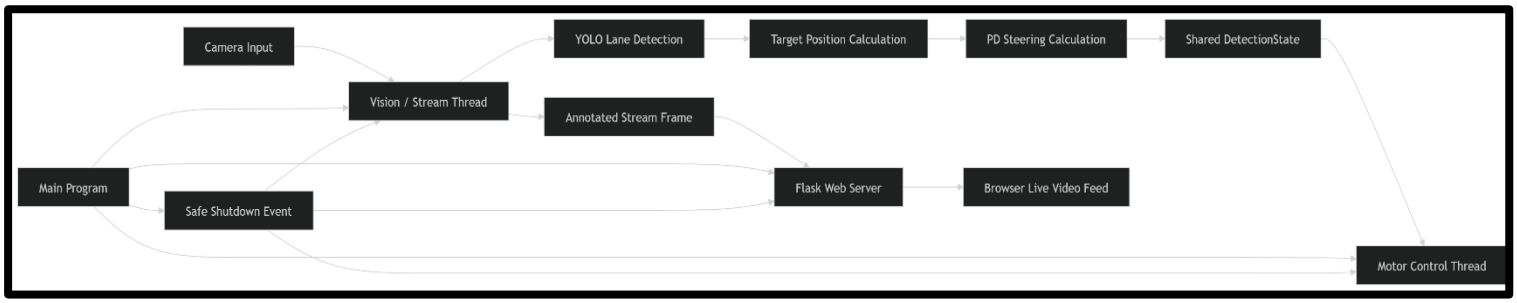

Figure 2. Implementation Script Structure Diagram

Results

During final testing, the Goosebot completed three laps in 45 seconds. The robot moved smoothly throughout the run and did not appear overly jerky or unstable. It maintained a steady speed while responding well to the detected lane markings.

One of the strongest results was the robot's ability to stay in the lane. The Goosebot remained within the proper lane for most of the run – slightly passing the center line occasionally - and made smooth steering corrections when needed. It also handled sharp turns in the corners well, with controlled fluid turning behavior rather than sudden or aggressive movements.

Some corners were more difficult for the robot to detect clearly. This may have been caused by the people moving around the track, glaring from the background windows, and imperfect detection under certain lighting conditions as the room had many people moving about. There are lots of variables that could've caused these issues which could be narrowed down with more targeted testing. Even with these issues highlighted, the Goosebot completed the laps successfully and remained in the proper lane consistently.

Discussion

The final results show that simplifying the model to focus only on white and yellow lane markings was effective for the competition environment. Since the final task mainly required lane-following, removing unnecessary obstacle-related classes from the model helped the AI focus on the most important features for steering. The reduced number of categories to detect could have potentially led to faster availability of the data needed to calculate the robot's movement, resulting in the smooth driving seen from the tests and the final video. Also contributing to the responsive and smooth driving of the Goosebot could have been the separation of the major tasks into a number of threads (two) manageable by the operating system of the Radxa Rock 5C, allowing these to run simultaneously and eliminate blocking between tasks.

The RKNN conversion process also improved the practicality of the final system. By converting the YOLO11 model into RKNN format using instructor Hoan Ngo's instructions in the "05_npu_conversion" section of the reference GitHub repository, the model became suitable for execution on the Rock 5C's NPU [1]. This helped support real-time detection, imperative for the speed requirements.

Conclusions

Overall, the Goosebot met the goal of completing autonomous lane-following using a YOLO11-based detection model, RKNN conversion, and a real-time motor control on the Radxa Rock 5C. During testing, the robot completed three laps in 45 seconds while maintaining smooth movement and strong lane position. Its ability to stay within the lane and handle the sharp turns with controlled motion showed that the final perception and steering setup worked effectively for the track environment.

Although the robot performed well, there are still areas where the design could be improved. Some detection issues occurred near corners, likely due to lighting changes, glare from the background window, moving people, or imperfect lane visibility. Future work should include training the model with more images from difficult track sections and varied lighting conditions so the robot can respond more consistently in challenging environments. On the model training front, another area to explore would be object segmentation, instead of object detection with bounding boxes, since segmentation can separate the objects clearly.

Mechanical improvements could also make the robot more reliable. Using wheels with better traction meant for the track could improve turning consistency and reduce slipping during sharp turns at higher speeds. Better wire management would help prevent loose connections and reduce clutter around the chassis. A redesigned frame could also provide better mounting locations for the battery, motor drivers, camera, and the Rock 5C, making the robot easier to maintain and more stable during its operation.

Furthermore, because of the above, the Goosebot demonstrated successful autonomous navigation with smooth motion, reliable lane-following, and solid performance. With improved mechanical organization, better wheels, cleaner wiring, and additional model trained for difficult visual conditions, the robot could become more consistent, durable, and competitive in future testing.

Acknowledgements

We would like to thank Dr. Ngo and Florida Polytechnic University for offering the opportunity to work on this project. As a group, we have gained extensive knowledge and hands-on experience of the process involved in autonomous driving, especially with training of detection models like YOLO 11.

References

Below are the links to the reference repository [1] and the project repository [2]

[1] Ngo, H. goose [Software]. https://github.com/hoanbklucky/goose

[2] Barreto, L.P., Beleut, M., & Gonzalez, F. (2026) goose-project [Software]. https://github.com/techyam08/goosebot-project/tree/main